

Case study on the analysis of SDLC models applied to Google Pay

Aim & Objectives

Aim:

To analyze how different Software Development Life Cycle (SDLC) models apply to **Google Pay (GPay)** and determine the optimal model for building a fast, secure, and reliable financial app.

Objectives:

- Evaluate traditional and modern SDLC models in the context of mobile payments.
- Analyze core modules of GPay (Customer, Merchant, Security & Fraud, Admin/Compliance).
- Compare the models based on security patching, regulatory changes, and system uptime.
- Recommend the best methodology for continuous operation and zero-downtime updates.

Selected Application Overview (Google Pay)

Google Pay is a digital wallet and instant payment platform linked directly to banking infrastructure (such as UPI). It requires **24/7 continuous availability**, **bank-grade encryption**, **instant execution**, and **real-time fraud detection**.

Functional Requirements:

- **Customer Module:** Registration, biometric/PIN login, linking bank accounts via UPI/cards, P2P money transfers, bill payments, QR scanning, and reward cashbacks.
- **Merchant Module:** Business profile verification, static/dynamic QR code generation, transaction tracking, daily settlements, and refund management.
- **Security & Fraud Module:** End-to-end data encryption, automated real-time fraud flags, and multi-factor authentication (MFA).
- **Admin & Compliance Module:** Helpdesk dispute handling, regulatory compliance monitoring, and banking gateway health checks.

SDLC Models Analysis for GPay

SDLC Model	Suitability	Key Advantages	Limitations for GPay
------------	-------------	----------------	----------------------

Waterfall	Low	Simple structure with clear, sequential phases.	Cannot accommodate sudden banking compliance updates or immediate security fixes once phase is locked.
Prototype	Medium	Great for refining user interface and screen flows.	Focuses on visual presentation rather than backend transaction security, scale, and database efficiency.
Spiral	High	Strong focus on early risk analysis and security testing.	High operational cost and slow release cycles for simple, routine app updates.
Agile	Very High	Enables small, fast feature updates and rapid reaction to feedback/bugs.	Requires a highly skilled team and active coordination to prevent feature creep.
DevOps	Very High	Automates security testing and deployments with zero service downtime.	Demands complex server infrastructure and continuous compliance monitoring.

Recommended Approach: Agile Combined with DevOps

Why Agile + DevOps is Essential for GPay:

- **Changing Regulations:** Banking rules and payment guidelines change frequently; Agile allows immediate sprint adjustments.
- **Immediate Security Patches:** Critical security vulnerabilities can be identified, patched, and deployed automatically via DevOps without taking the payment network offline.
- **Phased (Canary) Releases:** New features or UI updates can be deployed gradually to a small percentage of users to verify stability before full rollout.

Workflow:

Requirement Gathering (*New Bank Rule/Feature*) $\rightarrow$ **Sprint Planning** $\rightarrow$ **Secure Design** $\rightarrow$ **Development** $\rightarrow$ **Automated Security Testing** $\rightarrow$ **Gradual Deployment** $\rightarrow$ **Real-Time Monitoring** $\rightarrow$ **Next Sprint**

Detailed SDLC Phases for GPay

- **Requirement Analysis:** Collecting government regulations, bank API specs, and user feature requests. (*Deliverable: SRS Document*)
- **System Design:** Creating secure microservice architectures, cryptographic key management, and database schemas. (*Deliverables: Architecture & API Specs*)
- **Development:** Writing secure front-end (Kotlin/Swift) and backend (Java/Go) modules.
- **Testing:** Executing automated unit, load, penetration, and bank-gateway integration testing.
- **Deployment:** Phased releases through app stores and cloud infrastructure updates with zero downtime.
- **Maintenance:** 24/7 transaction monitoring, live fraud mitigation, and continuous performance tuning.

Example Test Cases

Test Case Scenario	Expected Result
Scan a valid merchant QR code	App fetches merchant details and prompts user for amount input.
Enter incorrect UPI PIN	Transaction is declined immediately with a clear error prompt.
Send money with insufficient balance	App notifies user of insufficient bank funds before initiating processing.
Check account balance	Securely fetches and displays current balance directly from the linked bank.

Comparison Summary

Criteria	Waterfall	Prototype	Spiral	Agile	DevOps
Requirement Flexibility	Poor	Good	Excellent	Excellent	Excellent

Security Patching Speed	Very Slow	Slow	Medium	Fast	Very Fast
Risk Management	Low	Medium	High	Medium	High
Delivery Speed	Slow	Medium	Medium	Fast	Very Fast
Development Cost	Low	Medium	High	Medium	Medium
Suitability for GPay	No	Partially	Yes	Best	Best

Results and Conclusion

- **Results:** High-stakes financial platforms like Google Pay require constant adaptation, automated testing, and zero downtime. Rigid traditional models fail to deliver the speed and security demanded by millions of daily financial transactions.
- **Conclusion:** Combining **Agile methodology** (for iterative, flexible development) with **DevOps practices** (for continuous testing and automated zero-downtime deployment) represents the gold standard for developing and sustaining Google Pay.